

第4章 函数与方法

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握方法声明与 C 函数的对照关系,理解 static 方法与实例方法的区别
- 彻底理解"Java 只有值传递":基本类型与引用类型如何传参,为什么 swap 写不出来
- 掌握方法重载与可变参数 varargs,理解重载解析规则
- 掌握递归与调用栈的关系,会写终止条件清晰的递归
- 掌握 lambda 表达式与函数式接口,会用方法引用简化代码
- 掌握 Stream API 的基本用法:过滤、映射、归约
- 建立"小函数、纯函数"的工程习惯,为第5章内存管理铺路

💡 从 C 到 Java:本章主题如何迁移

C 的函数是"全局自由函数",Java 的方法必须活在类里——这是语法差异,不是哲学差异:你照样可以把类当成"装函数的盒子"(工具类模式),把 static 方法当成 C 函数用。

真正的分水岭是参数传递。C 里传数组等于传指针(退化),传结构体等于复制;Java 里"数组和对象"永远传引用、基本类型永远传值,而且——引用也是按值传的。理解"传引用但是值传递"这句话,是理解整个 Java 参数体系的关键。

重载是 C 没有的能力。C 的 `printf` 用格式串魔法解决"多种参数形态";Java 用重载在编译期解决:同名方法、不同参数列表,编译器按参数精确选择。`printf` 的魔法在 Java 里变成了 `System.out.printf`,但生产代码更常用重载。

函数式编程是 C 程序员最大的增量。C 有函数指针,但回调写起来繁琐;Java 8 的 lambda 让"把行为当参数传递"成为日常,Stream API 则把"循环遍历 + 条件判断"改写成一行声明式代码。本章目标是:能读、能写、能辨别何时用 Stream。

学习顺序建议:先对照 C 的语法差异(\$4.1-4.3),再理解递归与调用栈(\$4.4),最后攻克 lambda 与 Stream(\$4.5-4.6)——前 60% 内容是"老朋友新语法",后 40% 才是 Java 的独家武器。

📁 前置知识自查

- C 函数:声明、定义、形参实参、返回值、递归、函数指针
- C 数组退化与指针传参的语义(实参是数组时到底传了什么)
- 第1章知识:基本类型、数组、字符串;第2章:运算符与表达式
- 第3章:流程控制(循环、分支)——Stream 之前必须熟练

🚀 本章学习路线

1. **第一阶段(约 1.5 小时):**\$4.1-4.3 与 C 对照着读,重点做"swap 陷阱"实验,彻底理解值传递
2. **第二阶段(约 1.5 小时):**\$4.4 递归 + \$4.5 lambda,每个示例敲一遍,体会"行为参数化"
3. **第三阶段(约 1.5 小时):**\$4.6 Stream 对照"循环写法"和"流式写法"逐段运行,再完成章末实验
4. **验收标准:**不看资料写出:swap 失败的完整解释、用 lambda 排序、用 Stream 统计平均分

本章知识导图

- §4.1 方法与 C 函数:声明对照 / static / main 签名
- §4.2 值传递:基本类型传值 / 引用传值 / swap 陷阱
- §4.3 重载与可变参数:重载解析 / varargs 规则
- §4.4 递归与调用栈:终止条件 / 栈帧概念
- §4.5 lambda 与方法引用:函数式接口 / 捕获
- §4.6 Stream API 入门:创建 / 中间操作 / 终止操作
- §4.7 方法设计工程实践:小函数 / 纯函数

§4.1 方法与 C 函数:声明对照

4.1.1 语法对照

主题	C 的写法	Java 的写法
顶层函数	int add(int a, int b) { ... }	必须放类里;static int add(int a, int b)
返回 void	void f() { }	void f() { }(相同)
声明位置	头文件声明 + 源文件定义	方法都定义在类体里,没有头文件
调用	f(x)	f(x)(同包/同文件);其他类用 类名.f(x)
程序入口	int main(int argc, char* argv[])	public static void main(String[] args)
参数个数可变	printf(...) 用变参宏 va_list	varargs:int... nums(语法糖)
同名函数	不允许同名(除非宏)	允许重载:同参数名不同参数列表

示例 4-1 工具类模式:把 C 函数装进类里

```
public class MathUtils {
    // static 方法:相当于 C 的全局函数,不需要对象即可调用
    static int add(int a, int b) {
        return a + b;
    }

    static int max3(int a, int b, int c) {
        return Math.max(Math.max(a, b), c); // 标准库 Math 也是工具类
    }

    public static void main(String[] args) {
        System.out.println(MathUtils.add(2, 3)); // 类名.方法名
        System.out.println(max3(1, 9, 4));      // 同类内可直接调用
    }
}
```

4.1.2 main 的签名

入口方法必须一字不差:public static void main(String[] args)。与 C 的 main(int argc, char* argv[]) 对照:args 就是 argv, args.length 相当于 argc(不含程序名),参数从 args[0] 开始。public 表示 JVM 可以从外部调用它;static 表示无需创建对象。

⚠ 易错点 1:main 写错签名的表现

签名写错(比如漏 `static`、把 `args` 写成别的)不会编译报错——JVM 只是找不到入口,运行时报 `Error: Main method not found`。C 里签名写错是编译警告 + 运行崩溃,Java 是"编译通过、运行报错",更隐蔽。写模板时逐字对照:`public static void main(String[] args)`。

§4.2 值传递:Java 最重要的参数语义

4.2.1 一句话结论

Java 只有值传递(`pass-by-value`):实参的值会被复制一份给形参。基本类型复制数值,引用类型复制"引用的拷贝"(两个引用指向同一个对象)。C 程序员最容易误解的点:传对象进去"改得动"(因为引用指向同一对象),但这不叫引用传递——形参本身还是实参的拷贝,在方法里给形参重新赋值,影响不到实参。

示例 4-2 值传递演示:swap 为什么失败

```
public class PassByValueDemo {
    public static void main(String[] args) {
        int a = 3, b = 7;
        swap(a, b);           // 传的是数值拷贝
        System.out.println(a + ", " + b);    // 3, 7:没有交换!

        int[] arr = {10, 20};
        swapArr(arr, 0, 1);    // 数组是引用,能改数组内容
        System.out.println(arr[0] + ", " + arr[1]); // 20, 10:交换了

        String s = "hello";
        changeRef(s);          // 形参被重新赋值,实参不变
        System.out.println(s); // hello:引用拷贝,改形参不影响实参
    }

    // C 里要传 int* 才能交换;Java 基本类型无法交换,只能返回或包装
    static void swap(int x, int y) {
        int t = x;
        x = y;
        y = t;                 // 只改了形参的拷贝
    }

    // 数组是引用:直接修改元素,影响"外面的"数组
    static void swapArr(int[] arr, int i, int j) {
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }

    static void changeRef(String str) {
        str = "world";        // 形参指向新字符串,实参纹丝不动
    }
}
```

传参形态	C 的语义	Java 的语义
基本类型	传值(拷贝)	传值(拷贝,相同)
数组	退化为指针,传地址;可改内容	传引用(引用的拷贝),可改内容;形参重新赋值不影响实参
结构体/对象	默认传值(整个复制);传指针才可改	永远传引用拷贝;可改字段,不能换引用
交换两个变量	传指针 swap(int* a, int* b)	无法实现(只能包装进数组/类或用返回值)
NULL/空值	指针可为 NULL	引用可为 null;使用前判空

★ 重点:一张图理解引用传递

把"引用"想成一根线:线头(变量)和线尾(对象)。传参 = 复制一根新线头,两端都拴着同一个对象。用新线头"拉回一个对象"(重新赋值),老线头不受影响;用新线头"改造对象"(改字段/改数组元素),对象变了,老线头看到的是同一个被改过的对象。这个心智模型可以解释 90% 的传参困惑。

⚠ 易错点 2:把"对象可修改"误读为"引用传递"

看到 `f(p)` 里改了 `p` 的字段、外面也变了,就以为 Java 是引用传递——错。测试法:在方法里执行 `p = new ...;`,如果外面跟着变,才是引用传递;Java 外面不变,所以是值传递(传的是引用的拷贝)。面试高频题,务必能说清。

§4.3 方法重载与可变参数

4.3.1 重载:同名不同参

同一类中可以有多个同名方法,参数列表不同(数量、类型、顺序)即构成**重载**。编译器根据实参类型在编译期选择目标方法。**返回值类型不参与重载判定**(只改返回值不叫重载,是编译错误)。常见用途:printf 风格的多形态输出、构造器重载(第6章)。

示例 4-3 重载示例

```
public class OverloadDemo {
    static void report(String name) {           // 一个参数
        System.out.println("学生:" + name);
    }

    static void report(String name, int score) { // 两个参数:重载
        System.out.println("学生:" + name + ",分数:" + score);
    }

    static void report(String name, double avg) { // 类型不同:重载
        System.out.println("学生:" + name + ",平均:" + avg);
    }

    // 返回类型不同不算重载!下面这行编译错误:
    // static int report(String name) { return 0; }

    public static void main(String[] args) {
        report("张三");           // 选 1 参版本
        report("李四", 88);       // 选 int 版本
        report("王五", 92.5);     // 选 double 版本
    }
}
```

4.3.2 可变参数 varargs

`int... nums` 表示"0 到任意个 int",底层是一个数组:`int... nums` 等价于 `int[] nums` (调用处自动包装)。规则:可变参数必须是最后一个参数;重载时可变参数是"兜底",编译器优先选精确匹配。

示例 4-4 varargs 示例

```
public class VarargsDemo {
    // 求任意个数的最大值:int... 等价于 int[]
    static int maxOf(int... nums) {
        if (nums.length == 0) {
            throw new IllegalArgumentException("至少需要一个数");
        }
        int max = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > max) {
                max = nums[i];
            }
        }
        return max;
    }

    // 可变参数必须是最后一个
    static void log(String tag, String... items) {
        System.out.print "[" + tag + " ] ";
        for (String item : items) {
            System.out.print(item + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        System.out.println(maxOf(3, 9, 1)); // 9
        System.out.println(maxOf()); // 抛异常(0 个参数)
        log("INFO", "启动", "连接", "完成"); // 传 3 个
        log("WARN"); // 传 0 个
        // 也可以直接传数组:
        int[] arr = {5, 6, 7};
        System.out.println(maxOf(arr)); // 7
    }
}
```

⚠ 易错点 3:varargs 与重载的"歧义陷阱"

`f(int...)` 与 `f(int[])` 同时存在会编译失败(相同签名); `f(int)` 与 `f(int...)` 同时存在时, `f(3)` 优先选精确的 `f(int)` ——但阅读代码的人常误以为会进 `varargs`。生产建议: `varargs` 版本只用于"确定是可变场景",不要与同语义的单参版本并存。

§4.4 递归与调用栈

4.4.1 递归与栈帧

递归与 C 完全相同:方法调用自己,每次调用在调用栈上分配一个**栈帧**(局部变量、参数、返回地址)。递归必须满足两件事:①有终止条件(基准情形);②每次递归都向基准情形靠近。缺一条就是栈溢出(StackOverflowError,第5章详讲内存)。

示例 4-5 递归:二分查找与斐波那契

```

public class RecursionDemo {
    // 二分查找(递归版):要求 arr 升序;返回下标,找不到返回 -1
    static int binarySearch(int[] arr, int target, int lo, int hi) {
        if (lo > hi) {
            return -1; // 终止条件:区间为空
        }
        int mid = lo + (hi - lo) / 2; // 防溢出的经典写法
        if (arr[mid] == target) {
            return mid; // 命中
        }
        if (arr[mid] > target) {
            return binarySearch(arr, target, lo, mid - 1); // 向左半区递归
        }
        return binarySearch(arr, target, mid + 1, hi); // 向右半区递归
    }

    // 斐波那契(递归版,指数复杂度,仅演示;生产用迭代)
    static long fib(int n) {
        if (n <= 1) {
            return n; // 基准情形
        }
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        int[] a = {1, 3, 5, 7, 9, 11};
        System.out.println(binarySearch(a, 7, 0, a.length - 1)); // 3
        System.out.println(binarySearch(a, 8, 0, a.length - 1)); // -1
        System.out.println(fib(10)); // 55
        // fib(50) 会非常慢:每层递归分裂成两个调用,总调用数接近 2^n
    }
}

```

★ 重点:递归改迭代的判定

递归深度与数据规模成正比时,生产代码优先改迭代(尾递归在 Java 中无优化)。判定方法:递归深度是 $O(n)$ 且 n 可能上万 → 改迭代;深度是 $O(\log n)$ (二分、快速排序) → 递归放心用。栈大小默认约 512KB~1MB,几万帧就能打满(第5章)。

§4.5 lambda 表达式与方法引用

4.5.1 函数式接口:lambda 的"类型"

lambda 必须挂在[函数式接口](#)上——只有一个抽象方法的接口(接口概念第6章详讲,现在只需会用)。Java 自带一批常用函数式接口: `Runnable` (无参无返回)、`Consumer<T>` (一个参数无返回)、`Function<T,R>` (一进一出)、`Predicate<T>` (返回 boolean)、`Supplier<T>` (无参出值)。

示例 4-6 lambda 的四种形态

```

import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Predicate;

public class LambdaDemo {
    public static void main(String[] args) {
        // 形态 1:无参数,一句话,等价于 Runnable
        Runnable task = () -> System.out.println("任务执行");

        // 形态 2:一个参数,省略括号和类型
        Consumer<String> printer = name -> System.out.println("你好," + name);

        // 形态 3:多语句用花括号,必须写 return
        Function<Integer, String> describe = n -> {
            if (n >= 60) {
                return "及格";
            }
            return "不及格";
        };

        // 形态 4:Predicate 返回 boolean,常用于过滤
        Predicate<Integer> isEven = n -> n % 2 == 0;

        task.run();
        printer.accept("张三");
        System.out.println(describe.apply(75));
        System.out.println(isEven.test(8));    // true

        // 用 lambda 给数组排序(Comparator 是函数式接口)
        String[] words = {"pear", "apple", "banana", "kiwi"};
        java.util.Arrays.sort(words, (x, y) -> x.length() - y.length());
        System.out.println(java.util.Arrays.toString(words));
        // [kiwi, pear, apple, banana] 按长度升序
    }
}

```

4.5.2 方法引用:已有方法的"简写指针"

lambda 体若只是"调用一个现有方法",可以缩写为**方法引用**: `类::静态方法`、`对象::实例方法`、`类::实例方法`、`类::new` (构造器引用)。它相当于 C 里"函数指针的地址",但类型安全、由编译器检查签名。

示例 4-7 方法引用示例

```

import java.util.Arrays;

public class MethodRefDemo {
    static int byLength(String a, String b) {
        return a.length() - b.length();
    }

    public static void main(String[] args) {
        String[] names = {"Tom", "Alice", "Bob", "Carolyn"};
        // lambda 写法:
        Arrays.sort(names, (a, b) -> byLength(a, b));
        // 方法引用写法(完全等价):
        Arrays.sort(names, MethodRefDemo::byLength);

        System.out.println(Arrays.toString(names));
        // 输出流式打印:println 是 System.out 的实例方法
        Arrays.asList(names).forEach(System.out::println);
    }
}

```

🚀 工程建议:lambda 的可读性红线

lambda 体超过 3 行就提取成具名方法,再使用方法引用。好处:方法有名字(自文档)、可单测、可复用。一行 lambda 写清,两行勉强,三行以上是坏味道。这与 C 里"回调逻辑复杂就写具名函数"是同一纪律。

§4.6 Stream API 入门

4.6.1 三件套:创建 → 中间操作 → 终止操作

Stream 是对数据集合的声明式流水线:① 创建(从数组/集合/范围);② 中间操作(过滤 filter、映射 map、排序 sorted,惰性、可链式);③ 终止操作(collect、forEach、count,触发真正计算)。没有终止操作,中间操作什么都不做。Stream 不修改原数据,返回新流。

示例 4-8 循环写法 vs 流式写法

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamDemo {
    public static void main(String[] args) {
        List<Integer> scores = Arrays.asList(45, 88, 62, 91, 55, 73, 100);

        // ---- C 程序员习惯的循环写法 ----
        int sum = 0, count = 0;
        for (int s : scores) {           // 遍历
            if (s >= 60) {               // 过滤
                sum += s;                // 映射(累加)
                count++;
            }
        }
        double avg = count == 0 ? 0 : (double) sum / count;

        // ---- Stream 写法:同样的逻辑,声明式 ----
        double avg2 = scores.stream()    // 创建
            .filter(s -> s >= 60)        // 中间:过滤及格
            .mapToInt(s -> s)            // 中间:转 int 流
            .average()                   // 终止:求平均
            .orElse(0);                  // 空流时给默认值

        System.out.println(avg + " vs " + avg2);    // 78.5 vs 78.5

        // 常用组合:过滤 + 映射 + 收集成新列表
        List<String> top = scores.stream()
            .filter(s -> s >= 90)
            .sorted((a, b) -> b - a)      // 降序
            .map(s -> "优:" + s)
            .collect(Collectors.toList());
        System.out.println(top);          // [优:100, 优:91]

        // 范围流:IntStream.range 是"带下标的增强 for"的替代
        int total = java.util.stream.IntStream.range(1, 101).sum();
        System.out.println(total);        // 5050
    }
}
```

★ 重点:惰性求值

中间操作不会立刻执行,只有终止操作被调用时才"拉取"元素逐个流过流水线——所以 `stream().filter(...).map(...)` 不接终止操作时,日志里一个元素都不会处理。这个"声明而不执行"的模型,是函数式编程与命令式循环最大的思维差异。

4.6.2 何时用 Stream:两条经验法则

- 处理"整个集合"且逻辑是"过滤-变换-汇总"三步之内 → Stream,代码更短、意图更显式
- 需要中途 break/带下标/多循环变量 → 传统循环,Stream 的短路(anyMatch、findFirst)不够直白时别硬拗

⚠ 易错点 4:Stream 不能复用

Stream 是"一次性"的:终止操作执行后流就关闭了,再调用会抛 `IllegalStateException: stream has already been operated upon`。需要再遍历就重新创建流。这与 C 里"数组可以反复遍历"的习惯不同,是新手最常见的 Stream 运行时错误。

§4.7 方法设计工程实践

📄 生产级方法设计清单

- **小而专**:一个方法只做一件事(通常 5-20 行);超过 40 行必须拆分。C 的"巨型函数"坏味道在 Java 里同样成立
- **纯函数优先**:不修改参数、不依赖全局可变状态,同样的输入必有同样的输出——便于测试与并行(第9章)
- **命名动词化**:calculateAverage、findById、isValid;boolean 返回用 is/has/can 前缀
- **参数宁少勿多**:超过 3 个参数考虑"参数对象"(第6章 records 是完美载体)
- **防御式返回**:返回 null 前想清楚:调用方 10 行外就要判空(第14章 Optional 解决)
- **文档注释**:public 方法写 javadoc 说明意图与参数约束,不是复述代码

本章速查表

主题	要点 / 写法
方法声明	static 返回类型 名(参数列表) { };工具类模式 = 类装 static 方法
入口	public static void main(String[] args);args 即 argv,args.length 即 argc
值传递	只有值传递;基本类型传拷贝,引用传引用的拷贝;方法内重新赋值不影响实参
swap	Java 无法交换两个基本类型变量;用返回值或包装数组
重载	同名不同参(数量/类型/顺序);返回值不参与判定
varargs	int... nums ≡ int[] nums;必须放最后;空参数时 length 为 0
递归	必须满足:终止条件 + 向基准靠近;深度 O(n) 且 n 大时改迭代
函数式接口	只有一个抽象方法的接口;Runnable/Consumer/Function/Predicate/Supplier
lambda	(参数) -> 表达式 或 { 多语句 };捕获变量须 effectively final
方法引用	类::静态方法 / 对象::方法 / 类::实例方法 / 类::new
Stream 三件套	创建 stream() → 中间 filter/map/sorted → 终止 collect/forEach/average
惰性求值	没有终止操作,中间操作不执行;流一次性,不能复用
循环 vs Stream	整集合"过滤-变换-汇总"用 Stream;需要 break/下标用循环
方法设计	小而专 / 纯函数 / 动词命名 / 参数 ≤3 / javadoc

最小必会清单

- 能逐字写出 main 签名并解释 public/static/void/args 的含义
- 能画图解释"值传递":swap 失败、数组内容可改、形参重赋值无效
- 能写出 2 个重载方法并说明重载判定规则(返回值不参与)

- 能写出 `varargs` 方法并说出它与数组的关系
- 能写出递归二分查找,并能判断何时递归该改迭代
- 能写出 `lambda` 的四种形态并说出函数式接口的定义
- 能写出"过滤及格→求平均"的 `Stream` 流水线,并解释惰性求值
- 能说出方法引用的四种形式并举例

自测题

Q 1. 下面的输出是什么? `int a = 1; f(a); System.out.println(a);` 其中 `static void f(int x) { x = 99; }`

✓ 参考答案

输出 1。基本类型按值传递,x = 99 只改形参拷贝。想改实参只能用返回值接收、包装进数组/类。

Q 2. 判断对错:Java 是引用传递,因为方法里修改对象字段,调用方能看到变化。

✓ 参考答案

错。那是"引用指向同一对象"的效果。验证法:方法里写 `p = new Point(...)` 后调用方不受影响,说明传的是引用的拷贝——值传递。

Q 3. 以下哪个是合法重载?① `int f(int x)` 与 `double f(int x)`;② `void f(int x)` 与 `void f(long x)`;③ `void f(int... a)` 与 `void f(int[] a)`

✓ 参考答案

只有 ② 合法:参数类型不同。① 只改返回值,非法;③ `int...` 与 `int[]` 签名相同,非法(编译错误:duplicate method)。

Q 4. 递归求 $1+2+\dots+n$,写出来;并说明 $n = 100000$ 时会发生什么。

✓ 参考答案

```
static long sum(int n) {
    if (n <= 0) return 0;           // 终止条件
    return n + sum(n - 1);         // 向基准靠近
}
```

$n = 100000$ 时递归深度 10 万层,超出默认栈大小,抛 `StackOverflowError`。改用 `for` 或公式 $n*(n+1)/2$ 。判定口诀:深度 $O(n)$ 且 n 可能很大 → 迭代。

Q 5. 把这段循环改写成 `Stream`:统计数组里正数的个数。 `int c = 0; for (int x : arr) if (x > 0) c++;`

✓ 参考答案

```
long c = Arrays.stream(arr).filter(x -> x > 0).count();
```

先 `stream()` 创建,filter 中间操作筛正数,count 终止操作计数。注意 `count` 返回 `long`。

Q 6. 为什么 `stream().filter(...).map(...)` 没有输出?如何让它工作?

✓ 参考答案

缺少终止操作,中间操作惰性执行、什么都不做。加一个终止操作,如 `.collect(Collectors.toList())` 或 `.forEach(System.out::println)`,流水线才真正运行。流是一次性的,第二次用要重新创建。

章末实验题:成绩处理系统

实验 4:成绩处理系统

实验目标:编写一个成绩处理程序,综合运用本章全部知识点:方法定义与调用、值传递、重载、varargs、递归、lambda、方法引用、Stream。

实验要求:

- 任务 1:定义 `readScores()` 读取成绩存入数组,主方法调用它(覆盖 §4.1 方法定义与调用)
- 任务 2:定义 `swapScore(int[] arr, int i, int j)` 交换数组元素,并在方法里演示"试图直接交换两个 int 局部变量失败"(覆盖 §4.2 值传递)
- 任务 3:重载 `report(String name)` 与 `report(String name, double avg)` (覆盖 §4.3 重载)
- 任务 4:定义 `maxOf(int... nums)` 求最高分,以 varargs 方式调用(覆盖 §4.3 varargs)
- 任务 5:用递归实现二分查找,在排序后的成绩里找目标分(覆盖 §4.4 递归)
- 任务 6:用 lambda 实现降序排序、用方法引用打印及格名单(覆盖 §4.5 lambda/方法引用)
- 任务 7:用 Stream 计算及格平均分,并筛选出前 3 名(覆盖 §4.6 Stream)
- 任务 8:main 中按顺序调用各方法,控制台输出完整报告(覆盖 §4.7 方法组织)

实验环境:JDK 17+; javac GradeSystem.java 编译, java GradeSystem 运行。

第一步 写 main 骨架:数组 + 读入 + 各任务方法占位,编译通过再逐个实现

第二步 任务 2 的"swap 失败演示"与"数组元素交换成功"对照打印,加深值传递理解

第三步 任务 6-7 对照着写:先写循环版,再改写 Stream 版,体会两者等价

参考答案要点

```

import java.util.Arrays;

/** 成绩处理系统 — 第4章综合实验参考答案
 * 覆盖:方法定义/调用、值传递、重载、varargs、递归、lambda、方法引用、Stream
 */
public class GradeSystem {
    private static final int[] scores = {45, 88, 62, 91, 55, 73, 100, 68, 59, 82};

    public static void main(String[] args) {
        // 任务 2:值传递演示
        int a = 1, b = 2;
        swap(a, b); // 失败:基本类型传值
        System.out.println("swap(a,b) 后: a=" + a + ", b=" + b); // a=1, b=2
        swapScore(scores, 0, 1); // 成功:数组是引用,可改内容
        System.out.println("交换后前两项: " + scores[0] + ", " + scores[1]);

        // 任务 3:重载
        report("张三");
        report("张三", average(scores));

        // 任务 4:varargs
        System.out.println("最高分 = " + maxOf(scores));
        System.out.println("varargs 单独调用 = " + maxOf(3, 9, 1, 7));

        // 任务 5:递归二分(先排序)
        int[] sorted = scores.clone();
        Arrays.sort(sorted);
        System.out.println("二分查找 68 的下标 = " + binarySearch(sorted, 68, 0, sorted.length - 1));

        // 任务 6:lambda 排序 + 方法引用打印
        Integer[] sArr = Arrays.stream(scores).boxed().toArray(Integer[]::new);
        Arrays.sort(sArr, (x, y) -> y - x); // lambda:降序
        System.out.print("降序: ");
        Arrays.asList(sArr).forEach(System.out::print); // 方法引用
        System.out.println();

        // 任务 7:Stream 统计
        double passAvg = Arrays.stream(scores)
            .filter(s -> s >= 60)
            .average()
            .orElse(0);
        System.out.printf("及格平均分 = %.1f%n", passAvg);
        System.out.println("前 3 名: " + Arrays.stream(scores)
            .boxed()
            .sorted((x, y) -> y - x)
            .limit(3)
            .toList());
    }

    // 任务 2:两个 int 无法交换(值传递,只改形参拷贝)
    static void swap(int x, int y) {
        int t = x;
        x = y;
        y = t;
    }

    // 数组元素可以交换:引用指向同一数组
    static void swapScore(int[] arr, int i, int j) {
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }

    // 任务 3:重载(参数个数不同)
    static void report(String name) {
        System.out.println("学生:" + name);
    }
}

```

```

static void report(String name, double avg) {
    System.out.printf("学生:%s,平均分:%.1f%n", name, avg);
}

// 任务 4:varargs 求最大值
static int maxOf(int... nums) {
    int max = nums[0];
    for (int i = 1; i < nums.length; i++) {
        if (nums[i] > max) {
            max = nums[i];
        }
    }
    return max;
}

// 任务 5:递归二分查找(数组升序)
static int binarySearch(int[] arr, int target, int lo, int hi) {
    if (lo > hi) {
        return -1;
    }
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) {
        return mid;
    }
    if (arr[mid] > target) {
        return binarySearch(arr, target, lo, mid - 1);
    }
    return binarySearch(arr, target, mid + 1, hi);
}

// 辅助:平均值(空数组返回 0)
static double average(int[] arr) {
    return arr.length == 0 ? 0 : (double) Arrays.stream(arr).sum() / arr.length;
}
}

```

设计说明:任务 2 把"失败的 swap"与"成功的数组交换"并列打印,一次看懂值传递;任务 6 故意对照 lambda 与方法引用;任务 7 的 Stream 全部是标准 API,没有手写循环。扩展方向:成绩来自控制台输入(第3章菜单)、异常处理(第9章)、按学生对象分组(第6章)。

下一章预告:第5章 内存管理。方法在调用栈上执行,对象在堆上生存——JVM 的内存区域划分、垃圾回收原理、内存泄漏与调优,是"生产级"Java 的必修课。